기술경력서

이름: 김성식
생년: 1987년생
연락처: 010-4442-0224
이메일: dshn21@naver.com
GitHub: https://github.com/menaje
LinkedIn: https://www.linkedin.com/in/성식-김-12b58b224/
현 소속: (주)희림종합건축사사무소, AI 서비스 기획/개발 (2024.05 ~ 현재)

목차

1. 프로필 요약
2. 기술 스택
3. 프로젝트 상세
4. 기술적 전문성
5. 경력 타임라인
6. 교육 및 자격

프로필 요약

한 줄 소개

10년간 건설사업관리(CM) 현장에서 발견한 문제를 기술로 해결하며, 비IT 배경에서 독학으로 AI 에이전트 시스템 아키텍트로 성장한 개발자

핵심 가치

"문제가 기술을 이끈다" - 기술을 위한 기술이 아닌, 현장의 실제 문제 해결을 위한 시스템 설계

차별화 포인트

1. **도메인 전문성 + AI 기술**: 10년 CM 경력 기반의 문제 정의 능력
2. **독학 능력**: 비IT 배경에서 시스템 아키텍처, LLM 오케스트레이션까지 마스터
3. **실전 검증**: 이론이 아닌 현장에서 검증된 시스템 구축 경험
4. **프로덕트 사고**: 사용자 가치 중심의 설계 철학

현재 포지션

(주)희림종합건축사사무소 AI 서비스 기획/개발 (2024.05 ~ 현재)

기술 스택

AI/LLM 기술

전문 분야: 다중 에이전트 오케스트레이션, 컨텍스트 엔지니어링

	분야	기술/도구	숙련도	설명
1	**LLM API**	OpenAI API, Anthropic Claude, Google Gemini	⭐⭐⭐⭐⭐	LLM 프로덕션 레벨 사용
2	**다중 에이전트 오케스트레이션**	커스텀 프레임워크	⭐⭐⭐⭐⭐	독자적 방법론 구축
3	**컨텍스트 엔지니어링**	프롬프트 설계, 단계 분할	⭐⭐⭐⭐⭐	대용량 문서 생성 최적화
4	**RAG 시스템**	Vector DB, 문서 검색, 임베딩	⭐⭐⭐⭐	법령/기준 관리 시스템 구축

주요 성과:

- 한 번의 실행으로 20여 종의 보고서 및 슬라이드 자동 생성
- 다중 AI 에이전트 간 컨텍스트 공유 및 역할 분담 시스템 설계

프로그래밍 언어

	언어	숙련도	사용 경험	주요 프로젝트
1	**Python**	⭐⭐⭐⭐⭐	2년 (2023~)	제안서 AI, 희림 사이드 프로젝트, CM 업무관리 시스템
2	**TypeScript**	⭐⭐⭐	1개월 (2025~)	WPM 프로젝트 관리 도구
3	**JavaScript**	⭐⭐⭐	1개월 (2025~)	자동화 스크립트

프로그래밍 철학:

- 명확한 변수명과 함수명으로 자기 문서화(Self-documenting) 코드 작성
- 타입 안정성을 위한 TypeScript 적극 활용
- 비동기 처리를 통한 성능 최적화

백엔드 및 데이터베이스

	기술	숙련도	사용 경험	설명
1	**Node.js**	⭐⭐⭐	1개월	CLI 도구 개발
2	**Fastify**	⭐⭐⭐	1개월	제안서 AI 백엔드 (26개 API 엔드포인트)
3	**Supabase**	⭐⭐⭐⭐	6개월	PostgreSQL + RPC + Realtime 통합 활용

아키텍처 경험:

- RESTful API 설계 및 구현
- 계층형 아키텍처 (API → Service → Utils → DB)
- 이벤트 기반 아키텍처 (AsyncIO Queue)
- Database-first 설계 (스키마 우선 설계)

개발 도구 및 인프라

	도구	숙련도	사용 경험	설명
1	**Git**	⭐⭐⭐	1개월	Git Hook 개발, 브랜치 전략, 버전 관리
2	**CLI 개발**	⭐⭐⭐⭐	1개월	WPM CLI 도구 (CAC 프레임워크)
3	**Obsidian**	⭐⭐⭐⭐⭐	2년	문서 관리 시스템 구축
4	**OCR/Vision**	⭐⭐⭐⭐	1년	애플 Vision 모델, Gemini 멀티모달 모델 활용

도메인 전문성

	분야	경험	설명
1	**건설사업관리(CM)**	10년	프로젝트 관리, 공무 업무, 품질 관리
2	**VE (Value Engineering)**	5년	13개 프로젝트 퍼실리테이터, DB 구축
3	**건축구조**	5년	석사 학위, 47개 프로젝트 기술 검토

프로젝트 상세

1. 제안서 에이전트 (Proposal Agent)

기간: 2024.05 ~ 2025.11 (진행 중)
역할: 기획 및 개발 (회사 프로젝트)
팀 구성: 2인

프로젝트 개요

다중 AI 에이전트를 활용한 제안서 자동 작성 시스템. 한 번의 실행으로 20여 종의 보고서 및 제안서 슬라이드를 한번에 자동 생성하며, 컨텍스트 엔지니어링 기반의 고품질 문서 생성 체계를 구축.

기술 스택

- **언어**: Python
- **LLM**: Google Gemini API
- **프레임워크**: FastAPI
- **DB**: Supabase (PostgreSQL)
- **아키텍처**: 계층형 + 이벤트 기반 하이브리드

아키텍처 설계

전체 시스템 구조 (계층형 아키텍처):

```
Client Layer (HTTP/REST)
    ↓
API Layer (FastAPI, 26 Endpoints)
    ↓
Service Layer (16 Services)
    ↓
Utils Layer (10 Modules)
    ↓
Database Layer (Supabase, 15 Tables)
    ↓
External Services (Gemini API, File Storage)
```

핵심 설계 원칙:

1. **관심사의 분리**: 각 레이어가 명확한 책임
2. **느슨한 결합**: 이벤트 기반 통신
3. **높은 응집도**: 관련 기능을 단일 모듈에 집중
4. **확장성**: 비동기 처리로 높은 동시성 지원

주요 서비스:

- **AssetManagementService**: 자산 업로드/업데이트, 파일 처리, 버전 관리
- **OrchestrationService**: Run, Phase, Stage, Task의 전체 실행 흐름 제어
- **AnalysisServices**: 각 자산 유형별 AI 기반 분석
- **LLMModelService**: LLM 모델 선택 및 관리
- **RecoveryService**: 실패 및 멈춤 상태의 Run/Task 복구

이벤트 기반 아키텍처

9가지 이벤트 타입:

1. `TASK_COMPLETED`: Task 완료/실패 시
2. `ASSET_ANALYSIS_COMPLETED`: 자산 분석 완료 시
3. `STAGE_COMPLETED`: Stage 완료 시
4. `PHASE_COMPLETED`: Phase 완료 시
5. `RUN_STATUS_CHANGED`: Run 상태 변경 시
6. `SYSTEM_HEALTH`: 시스템 상태 변경 시
7. `PROJECT_INFO_EXTRACTION_REQUESTED`: 프로젝트 정보 추출 요청 시
8. `PROJECT_INFO_EXTRACTION_COMPLETED`: 프로젝트 정보 추출 완료 시
9. `ASSETS_READY_FOR_PLANNING`: 모든 자산 준비 완료 시

하이브리드 모드:

- 이벤트 기반 (실시간 반응성) + 폴링 (안정성 및 복구력) 병행

실행 엔진

계층적 실행 구조:

```
Run
├── Phase 1
│   ├── Stage 1
│   │   ├── Task 1
│   │   │   ├── Tool Task 1
│   │   │   └── Tool Task 2
│   │   └── Task 2
│   └── Stage 2
│       └── Task 3
└── Phase 2
    └── ...
```

의존성 관리:

- 위상 정렬 (Topological Sort)로 실행 순서 결정
- 병렬 실행 가능 Task 그룹화
- 순환 의존성 검사

핵심 성과

1. **자동화 성공**: 한 번의 실행으로 20여 종의 보고서 및 제안서 슬라이드 자동 생성
2. **품질 확보**: 컨텍스트 엔지니어링을 통한 일관성 및 전문성 유지
3. **확장성**: 계층형 + 이벤트 기반 아키텍처로 높은 확장 가능성
4. **복구 메커니즘**: 자동 재시도 및 복구 시스템 구축

기술적 도전 및 해결

도전 1: 20여 종 보고서 및 슬라이드 일관성 유지

- 문제: 다양한 형식의 보고서 및 슬라이드 생성 시 앞뒤 내용 불일치
- 해결: 단계별 작업 분할, 컨텍스트 공유 메커니즘, Phase별 검증

도전 2: 다중 AI 에이전트 조율

- 문제: 여러 AI 에이전트 간 역할 분담 및 협업
- 해결: 명확한 에이전트 역할 정의, 이벤트 기반 통신, 의존성 관리

도전 3: 실패 복구

- 문제: LLM API 호출 실패, 네트워크 오류
- 해결: 지수 백오프 재시도, RecoveryService, Health Check 시스템

학습 포인트

- **계층형 아키텍처**: 관심사 분리를 통한 유지보수성 향상
- **이벤트 기반 시스템**: 실시간 반응성과 안정성 균형
- **LLM 최적화**: 프롬프트 엔지니어링, 컨텍스트 윈도우 관리
- **에러 처리**: Fail-safe 설계

---

2. 개발자 프로젝트 관리 도구

기간: 2025.11 ~ 현재 (진행 중)
역할: 기획 및 개발 (개인 프로젝트)
팀 구성: 1인

프로젝트 개요

CLI 에이전트(클로드 코드, 제미나이 CLI, Codex)를 위한 작업 및 프로젝트 관리 CLI 시스템. Git과 통합하여 작업을 자동 추적하고, 다중 AI 에이전트 협업을 지원하는 지능형 워크플로우 도구.

기술 스택

- **언어**: TypeScript
- **런타임**: Node.js 18+
- **프레임워크**: CAC (CLI)
- **DB**: Supabase (PostgreSQL)
- **Git**: simple-git, Git Hook
- **Future**: Vector DB (시스템 맥락 관리 예정)

핵심 기능 (11개)

1. 작업 계획 라이프사이클 관리

- 작업 등록, 시작, 완료 추적
- 상태 전환: 대기 → 진행중 → 완료

2. 자동 Git 커밋 추적

- 포스트-커밋 훅 자동 설치
- 커밋 메시지에서 `#N` 태그 자동 감지

3. 세션 기반 시간 추적

- 브랜치별 독립적 세션 관리
- 개발자별, 작업 계획별 실제 작업 시간 측정

4. Git Identity 관리

- Git 설정에서 개발자 자동 식별
- 프로젝트별 identity (동일 이메일, 다른 프로젝트 = 다른 identity)

5. 세션 일관성 검증

- 6가지 비정상 상태 자동 감지
- 완료 상태 작업에 활성 세션 존재 등

6. 로비 및 후속 세션 모델

- 로비 세션: 작업 계획 전 탐색/토론 단계 추적
- 후속 세션: 완료 후 마무리 작업 시간 추적
- 4가지 세션 상태: lobby, active, completed, follow-up

7. 프로젝트 자동 감지

- 3개 경로 체계: 작업영역, Git 경로
- `.git` 폴더 자동 탐색

8. CLI 사용성

- `--json` 플래그로 JSON 출력 지원
- 일관된 명령 구조: `coni-wpm [options]

9. 문서 메타데이터 검색

- YAML Frontmatter 기반 문서 검색
- 타입별 검색 (`--type`), 키워드 검색 (`--keyword`)
- 역방향 조회 (`--references`)

10. 워크플로우 가이던스 및 검증 (Guardian)

- 워크플로우 검증 (중복 작업, 미커밋 변경사항)
- 경고 시스템: Warning (소프트 스톱), Info (비차단형), Error (하드 스톱)

11. 프로젝트 설정 파일 (.wpmrc.json)

- Glob 패턴 기반 문서 스캔 제외
- 프로젝트별 커스터마이징

아키텍처 설계

CLI 주요 도구 구조:

```
// CLI Commands
coni-wpm init           // Git Hook 설치
coni-wpm register       // 작업 등록
coni-wpm start          // 작업 시작 (세션 생성)
coni-wpm status         // 상태 조회
coni-wpm complete       // 작업 완료 (세션 종료)
coni-wpm query-docs     // 문서 검색
```

데이터베이스 스키마 (주요 테이블):

- `projects`: 프로젝트 정보
- `work_plans`: 작업 계획
- `work_sessions`: 작업 세션 (시간 추적)
- `work_commits`: Git 커밋 기록
- `git_identities`: 개발자 식별 정보

Git Hook 통합:

```
# .git/hooks/post-commit
#!/bin/bash
coni-wpm log-commit
# → WPM 오류 시에도 커밋 성공 (Fail-safe)
```

현재 진행 상황

완료된 기능:

- ✅ Git 커밋 기반 작업 자동 추적 시스템
- ✅ AI 에이전트 협업 프로토콜 설계
- ✅ 세션 기반 작업 시간 관리
- ✅ 자연어 스크립트 기반 다중 에이전트 오케스트레이션
- ✅ DB 구조 및 워크플로우 자동화
- ✅ 문서 메타데이터 검색 시스템
- ✅ Guardian 기능 (워크플로우 검증 및 안내)
- ✅ Glob 패턴 기반 설정 파일

진행 중:

- 🔄 Vector DB 통합 (시스템 맥락 관리)
- 🔄 지능형 작업 관리 및 추천 시스템

향후 계획

1. **Vector DB 활용 시스템 맥락 관리**
   - 작업 이력 임베딩 및 검색
   - AI 에이전트가 과거 결정을 참조하여 일관성 유지

    - 유사 작업 추천 시스템
2. **지능형 작업 관리 (Ontology Integration)**
    - 작업 간 관계 자동 파악
    - 의존성 자동 제안
    - 작업 우선순위 추천

핵심 성과

1. **개발 프로세스 표준화**: 7단계 워크플로우 정립
2. **자동 추적**: Git 커밋 자동 기록 (수동 입력 0%)
3. **시간 측정**: 세션 기반 정확한 작업 시간 추적
4. **AI 협업**: 클로드, 제미나이 간 협업 프로토콜 구축

기술적 도전 및 해결

도전 1: Git Hook Fail-safe

- 문제: WPM 오류 시 커밋 차단 위험
- 해결: 모든 에러 처리, stderr만 사용, 항상 exit 0

도전 2: 세션 일관성

- 문제: 비정상 세션 상태 (완료 작업에 활성 세션)
- 해결: 자동 일관성 검증 (6가지 패턴), DB 레벨 검증 함수

도전 3: 경로 체계

- 문제: 작업영역, Git 경로, 프로젝트 경로 혼란
- 해결: 3개 경로 명확히 분리, 자동 탐색, 투명한 출력

학습 포인트

- **CLI 도구 설계**: 사용자 경험 중심 명령어 설계
- **Git 통합**: Hook 메커니즘, 안전 장치
- **데이터 일관성**: DB 레벨 검증, 트랜잭션 관리
- **Fail-safe 설계**: 모든 실패 시나리오 대비

---

3. CM 업무관리 시스템

기간: 2023.08 ~ 2024.01
역할: 기획 및 개발 (1인)
프로젝트: 대전 5개현장 통합건설사업관리

프로젝트 개요

Obsidian 기반 맞춤형 문서관리 시스템. 5개 현장 통합 문서관리 및 프로젝트 데이터 관리 효율화. 애플 Vision 모델 및 Gemini 멀티모달 모델을 활용한 문서 자동 등록 및 요약 시스템 구축.

기술 스택

- **플랫폼**: Obsidian (Markdown 기반 지식 관리 도구)
- **언어**: Python
- **AI**: GPT API (OpenAI)
- **Vision/OCR**: 애플 Vision 모델 → Gemini 멀티모달 모델
- **자동화**: Python 스크립트

시스템 아키텍처

문서 자동 등록 파이프라인:

```
PDF 파일
    ↓
애플 Vision 모델 / Gemini 멀티모달 모델 (텍스트 추출)
    ↓
GPT API 요약
    ↓
Markdown 파일 생성
    ↓
Obsidian Vault 저장
```

데이터 관리:

- 사업비, 계약 데이터 자동 분석 및 시각화
- 공정, WBS 업무별 관리
- 이슈 트래킹 및 의사결정 기록

월간보고서 자동 생성:

- 템플릿 기반 보고서 구조
- 데이터 자동 집계 (사업비, 공정 진행률)
- GPT API를 활용한 요약 및 설명 생성

핵심 성과

1. **문서관리 효율화**: 5개 현장 통합 문서 중앙 관리
2. **자동화**: OCR + GPT를 통한 문서 자동 등록 및 요약
3. **시각화**: 사업비, 계약 데이터 실시간 대시보드
4. **협업 시스템**: 현장 전체 핵심 협업 시스템으로 자리매김

기술적 도전 및 해결

도전 1: PDF 문서 자동 처리

- 문제: 다양한 형식의 PDF 문서 (스캔본, 텍스트, 표)
- 해결: 애플 Vision 모델 및 Gemini 멀티모달 모델 + 정규표현식 조합, GPT를 활용한 구조화

도전 2: 대용량 데이터 관리

- 문제: 5개 현장의 방대한 문서 및 데이터
- 해결: Obsidian의 태그 및 링크 체계 활용, 검색 최적화

도전 3: 사용자 학습 곡선

- 문제: 비IT 사용자의 Obsidian 학습
- 해결: 간소화된 UI, 자동화로 입력 최소화

학습 포인트

- **문서 관리**: Markdown 기반 지식 관리 체계
- **Vision + AI**: 비정형 데이터의 구조화 (애플 Vision, Gemini 멀티모달)
- **자동화**: 반복 업무의 파이프라인 구축

---

4. 희림 사이드 프로젝트

기간: 2024.05 ~ 2024.12
역할: 기획 및 개발 (1인)

프로젝트 개요

건설 도메인 지식 관리를 위한 RAG 기반 시스템 모음. 법령 데이터 자동 수집, 건설관련 기준 검색, 건설기술 자율 지식 생성 시스템.

기술 스택

- **언어**: Python
- **RAG**: Retrieval-Augmented Generation
- **Vector DB**: Supabase Vector (pgvector)
- **API**: 공공데이터 API (법제처, 국가기술표준)
- **LLM**: OpenAI API, Gemini API

서브 프로젝트

1. 법령 데이터 자동 수집 및 RAG 시스템

- 법제처 API를 통한 건설 관련 법령 자동 수집
- 개정 사항 자동 감지 및 업데이트
- Vector DB 기반 의미 검색
- 질의응답 시스템 (RAG)

2. 건설관련 기준 벡터 DB 기반 검색 시스템

- 국가기술표준 API 연동
- 건설 기준, 지침 문서 수집
- 임베딩 및 벡터 DB 저장
- 유사 기준 검색 및 추천

3. 입력 파일에서 LLM 데이터 추출 및 관리

- PDF, Word 파일에서 구조화된 데이터 추출
- LLM을 활용한 메타데이터 생성
- 자동 분류 및 태깅

4. 건설기술 자율 지식 생성 시스템

- LLM을 활용한 기술 문서 자동 생성
- KMS(Knowledge Management System) 보완
- 지식 그래프 구축

핵심 성과

1. **자동화**: 법령 및 기준 데이터 자동 수집 및 업데이트
2. **검색 향상**: 벡터 DB 기반 의미 검색으로 정확도 향상
3. **지식 생성**: LLM을 활용한 자율 지식 생성 체계 구축

기술적 도전 및 해결

도전 1: 법령 데이터 구조화

- 문제: 비정형 법령 텍스트
- 해결: LLM을 활용한 구조 추출, 계층 구조 파싱

도전 2: 벡터 검색 정확도

- 문제: 일반 임베딩의 도메인 특화 부족
- 해결: Fine-tuning된 임베딩 모델 탐색, 검색 결과 재순위화

학습 포인트

- **RAG 시스템**: Vector DB 기반 검색 및 생성
- **공공 API 연동**: 데이터 수집 및 업데이트 자동화
- **도메인 특화**: 건설 분야 지식 관리 체계

---

5. VE 업무 자동화 시스템

기간: 2017 ~ 2019
역할: 기획 및 개발 (1인)
프로젝트: 13개 VE 프로젝트

프로젝트 개요

VE(Value Engineering) 업무 프로세스 표준화 및 전산화. Excel 기반 보고서 자동 생성 시스템 구축. VE 아이디어 및 아이템 데이터베이스 구축.

기술 스택

- **도구**: Excel (수식)
- **DB**: Excel 기반 데이터베이스

핵심 기능

1. **VE 업무 프로세스 표준화**
   - 기능 정의, 대안 생성, 평가 단계 표준화
   - 입력 양식 통일
2. **보고서 자동 생성**
   - 사용자가 최소한의 데이터 입력
   - 자동으로 표, 그래프, 요약 생성
   - VE 보고서 포맷 자동 적용
3. **VE 데이터베이스**
   - 과거 프로젝트의 아이디어 및 아이템 축적
   - 검색 및 필터링 기능
   - 재사용 가능한 아이디어 제안

핵심 성과

1. **효율화**: 13개 VE 프로젝트 수행 시간 단축
2. **품질 향상**: 표준화된 프로세스로 일관된 품질 유지
3. **지식 축적**: VE 아이디어 DB 구축으로 재사용성 증대

기술적 도전 및 해결

도전 1: Excel의 한계

- 문제: 복잡한 로직 구현 어려움
- 해결: 수식의 창의적 조합, 조건부 서식 활용

도전 2: 데이터 검색

- 문제: Excel의 검색 기능 제한
- 해결: VLOOKUP, INDEX/MATCH 조합, 필터 자동화

학습 포인트

- **업무 자동화**: 반복 작업의 자동화 가치
- **사용자 중심 설계**: 비IT 사용자를 위한 UI
- **점진적 개선**: 작은 자동화부터 시작하여 확장

기술적 전문성

다중 에이전트 오케스트레이션

핵심 개념

여러 AI 에이전트를 조율하여 복잡한 작업을 수행하는 시스템 설계. 각 에이전트에게 명확한 역할을 부여하고, 협업 프로토콜을 통해 일관된 결과물 생성.

독자적 방법론

"이론보다 실전" - 제안서 작성이라는 고차원적 문제를 해결하기 위해 치열하게 고민한 결과, 컨텍스트 엔지니어링과 다중 에이전트 오케스트레이션이라는 최신 기술의 핵심 원리와 일치하는 독자적 방법론을 구축.

핵심 원칙

1. **명확한 역할 분담**: 각 에이전트의 책임과 권한 정의
2. **컨텍스트 공유**: 일관성 유지를 위한 맥락 전달
3. **의존성 관리**: Task 간 순서 및 병렬 실행
4. **피드백 루프**: 결과 검증 및 재시도

실전 적용 (제안서 AI)

```
User Request
    ↓
OrchestrationService (전체 조율)
    ↓
Phase 1: 자산 분석
    ├─ Asset 1 Analysis Agent
    ├─ Asset 2 Analysis Agent
    └─ Asset 3 Analysis Agent (병렬 실행)
    ↓
Phase 2: 보고서 생성
    ├─ Stage 1: 개요 작성 Agent
    ├─ Stage 2: 상세 작성 Agent (순차 실행)
    └─ Stage 3: 검증 Agent
    ↓
Final Output (20여 종의 문서)
```

컨텍스트 엔지니어링

핵심 개념

LLM에 정확한 맥락을 제공하여 품질을 극대화하는 기법. 프롬프트 최적화, 단계별 작업 분할, 대용량 문서 생성 시 일관성 유지.

핵심 기법

1. **단계별 분할 (Chunking)**
   - 대용량 작업을 Phase → Stage → Task로 분할
   - 각 단계별 명확한 입력 및 출력 정의
2. **컨텍스트 누적 (Context Accumulation)**
   - 이전 단계의 결과를 다음 단계의 입력으로 전달
   - 일관성 유지를 위한 핵심 정보 추출
3. **프롬프트 최적화 (Prompt Optimization)**
   - 명확한 지시어 (Instruction)
   - 출력 형식 명시 (Format Specification)
4. **검증 및 재시도 (Validation & Retry)**
   - 결과 검증 로직
   - 실패 시 피드백을 통한 재시도

실전 예시 (A4 110페이지 문서 생성)

```
1. 전체 구조 계획 (Phase Planning)
   → 목차, 장별 목적 정의

2. 장별 상세 작성 (Stage Execution)
   → 이전 장의 맥락 전달
   → 일관된 용어 및 톤 유지

3. 검증 및 수정 (Validation)
   → 앞뒤 내용 일치 확인
   → 재작성 필요 시 피드백 제공
```

RAG (Retrieval-Augmented Generation) 시스템

핵심 개념

Vector DB 기반 문서 검색 및 생성. 대규모 지식 베이스에서 관련 정보를 검색하여 LLM에 제공, 정확하고 최신 정보 기반의 답변 생성.

아키텍처

```
User Query
    ↓
Query Embedding (임베딩 생성)
    ↓
Vector DB Search (유사도 검색)
```

```
    ↓
Retrieved Documents (상위 K개 문서)
    ↓
LLM Generation (검색 결과 + Query)
    ↓
Final Answer
```

실전 적용 (희림 사이드 프로젝트)

법령 RAG 시스템:

1. 법제처 API로 법령 수집
2. 법령 텍스트 임베딩
3. Vector DB 저장
4. 사용자 질의 → 유사 법령 검색
5. LLM으로 요약 및 해석 생성

성과:

- 검색 정확도 향상 (키워드 → 의미 검색)
- 최신 정보 자동 반영 (개정 법령 자동 업데이트)

시스템 아키텍처 설계

설계 철학

"데이터 중심 설계 (Data-Centric Design)" - 모든 혁신의 시작과 끝은 데이터. 잘 설계된 데이터 흐름은 그 자체로 강력한 솔루션.

핵심 원칙

1. **모듈형 구조 (Modular Architecture)**
   - 기능별 모듈 분리
   - 명확한 인터페이스 정의
   - 독립적인 배포 및 테스트
2. **확장 가능한 설계 (Scalable Design)**
   - 수평 확장 가능
   - 비동기 처리 (AsyncIO, Event-driven)
   - 연결 풀링 및 캐싱
3. **Git 통합 (Git Integration)**
   - Git Hook 활용
   - 커밋 기반 자동 추적
   - 브랜치 전략 통합

실전 적용

제안서 AI 아키텍처:

- 계층형 구조 (API → Service → Utils → DB)
- 이벤트 기반 통신 (9가지 이벤트)
- 의존성 주입 (FastAPI Depends)
- 재시도 메커니즘 (지수 백오프)

WPM 아키텍처:

- CLI 도구 설계 (CAC 프레임워크)
- Git Hook 통합 (Fail-safe)
- 2개 경로 체계 (작업영역, Git 경로)
- Guardian 기능 (워크플로우 검증)

경력 타임라인

타임라인 차트

```
2015 ──────────── 2019 ──────────── 2023 ──────────── 2025
 │                 │                 │                 │
CM 현장 + 기술팀   CM 현장 + 연구소   CM + 시스템 개발   에이전트 개발
 │                 │                 │                 │
 ├─ 세종시 CM       ├─ 삼일빌딩       ├─ 대전 5개현장     ├─ 희림 입사
 └─ 본사 기술팀     └─ 연구소         ├─ Obsidian 시스템  ├─ 제안서 AI
    ├─ 구조기술지원                    └─ GPT 활용         └─ 프로젝트관리 개발
    └─ VE
```

기술 성장 곡선

Phase 1 (2015~2019): 도메인 전문가

- 건설사업관리(CM) 실무
- VE 전문가 양성
- 엑셀 기반 자동화 시작
- **전환점**: "데이터를 활용한 의사결정"의 중요성 인식

Phase 2 (2019~2023): 시스템 경험

- 프로젝트 관리 및 협업 도구 도입 (Trello, REDMINE)
- **전환점**: "협업을 통한 프로젝트관리" 경험

Phase 3 (2023~현재): AI 시스템 아키텍트

- Obsidian 기반 문서관리 시스템 개발
- AI를 활용한 Python 독학 및 실무 적용

- Vision 모델 + GPT 통합
- 다중 에이전트 오케스트레이션 방법론 구축
- 제안서 AI 시스템 개발
- WPM 프로젝트 관리 도구 개발
- RAG 시스템 구축
- **현재**: "AI가 데이터를 이해하는 시대" 목격 + "데이터가 스스로 일하는 지능형 시스템" 설계

교육 및 자격

학력

충북대학교 건축공학과 석사 (2011.03 ~ 2013.02)

- 전공: 건축구조해석

충북대학교 건축공학과 학사 (2005.03 ~ 2011.02)

자격사항

- **건축기사**: 건축 구조 설계 및 검토
- **VE 고급과정 이수**: Value Engineering 전문가

독학 및 자기 학습

프로그래밍 (2023~):

- AI를 활용하여 Python, TypeScript, JavaScript 독학
- 공식 문서, 프로젝트 기반 학습
- AI 도구를 활용한 빠른 학습 및 실무 적용

AI/LLM (2023~):

- OpenAI, Anthropic, Google 공식 문서
- 실전 프로젝트를 통한 학습

시스템 아키텍처 (2023~):

- 클린 아키텍처, DDD 개념 학습
- 오픈소스 프로젝트 분석
- 실무 적용 및 피드백

프로필 마무리

나의 개발 철학

1. 문제가 기술을 이끈다
기술을 위한 기술이 아닌, 현장의 실제 문제 해결을 위한 시스템을 만듭니다. 10년간 CM 현장에서 경험한 '파편화된 데이터'와 '암묵지'라는 문제가 저의 모든 개발의 출발점입니다.

2. 데이터 중심 설계
모든 혁신의 시작과 끝은 데이터입니다. 잘 설계된 데이터 흐름은 그 자체로 강력한 솔루션이 됩니다. AI는 데이터라는 영토 위에서 스스로 추론하고 판단하는 지능형 에이전트입니다.

3. 실전 검증
이론이 아닌 실전에서 검증된 시스템을 만듭니다. 독학으로 습득한 지식을 즉시 프로젝트에 적용하며, 실패와 성공을 통해 배웁니다.

4. 사용자 가치 중심
단순한 자동화를 넘어, 사용자에게 진정한 가치를 제공하는 프로덕트를 만듭니다. 비IT 사용자도 쉽게 사용할 수 있는 직관적인 시스템을 지향합니다.